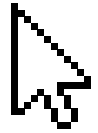
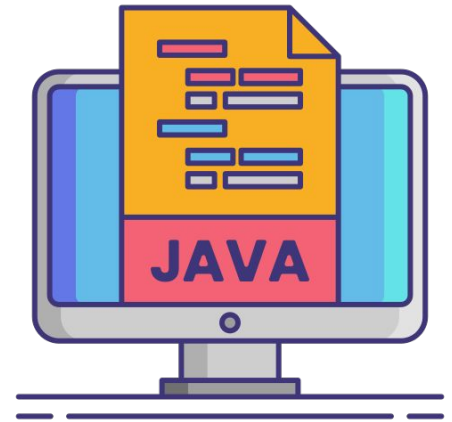




CISC 1115: Introduction to Programming Using Java



Summer 2026 Prep Workshop





Before We Begin

- In order to effectively engage in this workshop students will need the following:
 - **A working microphone (camera optional)**
 - **A stable internet connection**
 - **A folder on your computer for today's workshop files**
- By the end of today's workshop, you should have:
 - **Java Development Kit (JDK) installed**
 - **NetBeans IDE installed**
 - **Your first Java program running successfully!**
- If you get stuck, don't worry—we'll troubleshoot together. Programming is all about learning through experimentation, asking questions, and solving problems one step at a time. No prior Java experience is required!



Itinerary

- 01 Introduction to Java
- 02 Downloading and Installing the JDK
- 03 Writing Your First Java Program (Part 1 - Using the CLI)
- 04 Introduction to IDEs
- 05 Writing Your First Java Program (Part 2 - Using the IDE)
- 06 Overview and Next Steps



01

Introduction to java





What is Java?

- **Java** is a high-level, class-based, popular **object-oriented programming language**, created in 1995.
- It is owned by Oracle, and more than 3 billion devices run Java.
- The rules and syntax of Java are based on the C and C++ languages.
- It is used for:
 - Mobile applications (especially Android apps)
 - Desktop and Web applications
 - Web servers and application servers
 - Games
 - Database connection
 - And much, much more!



Why use Java?

- Java works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
- It is one of the most popular programming languages in the world
- It has a large demand in the current job market
- It is easy to learn and simple to use
- It is open-source and free
- It is secure, fast and powerful
- It has huge community support (tens of millions of developers)
- Java is an object oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs
- As Java is close to C++ and C#, it makes it easy for programmers to switch to Java or vice versa

Getting Started With Java

- To create an application using Java, you will need:
 - To **download and install the Java Development Kit (JDK)**, which is available for Windows, macOS, and Linux.
 - A **command-line interface (CLI)** on your computer that allows you to create and delete files, run programs, and navigate through folders and files (**On a Mac, it's called Terminal, and on Windows, it's Command Prompt**) OR (preferably) **an integrated development environment (IDE)** - a software application that helps programmers develop software code efficiently by combining capabilities such as software editing, building, testing, and packaging in an easy-to-use application, such as Eclipse, NetBeans, IntelliJ, DrJava, etc.



02



Downloading and Installing the JDK (Java Development Kit)



What is the JDK?

- The **Java Software Development Kit (JDK)** is a set of tools that are used to develop Java applications. It includes everything you need to compile, debug, and run Java programs.
- The JDK is an essential tool for Java programmers as it **provides the necessary libraries, executables, and tools to develop Java applications.**
- It includes:
 - the **Java Runtime Environment (JRE)**
 - **compiler (javac)**
 - **interpreter (java)**
 - and **other tools needed for Java development**

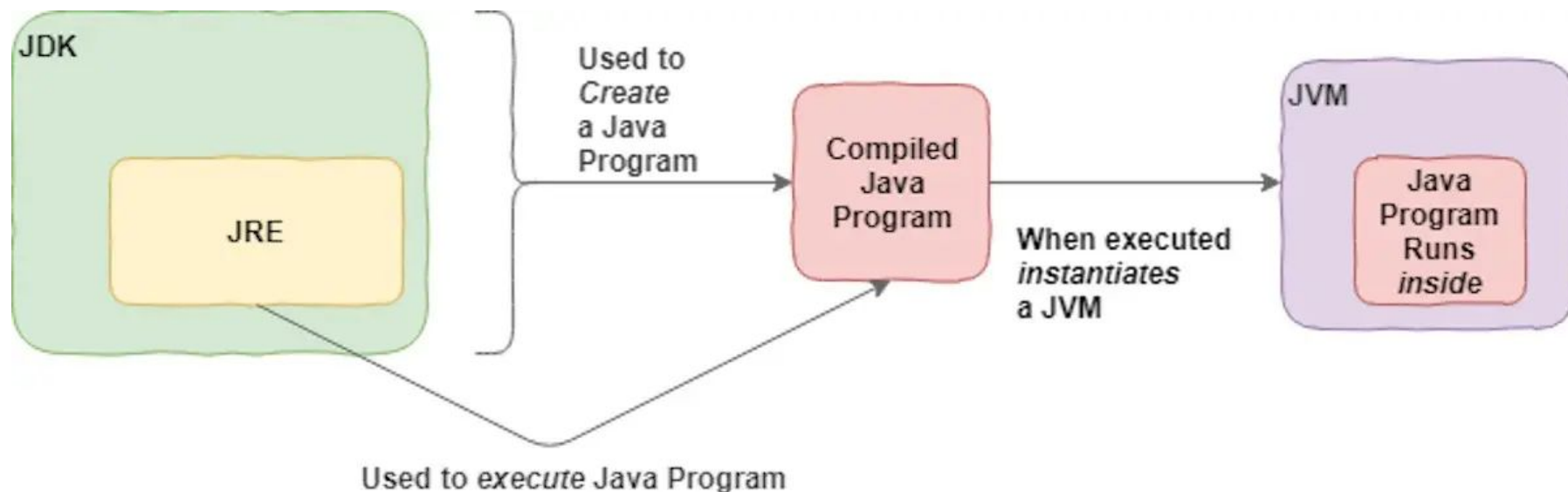


What is the JDK?

- The **JDK** is one of three core technology packages used in Java programming, along with the **JVM (Java Virtual Machine)** and the aforementioned **JRE**.
- It's important to differentiate between these three technologies and understand how they're connected:
 - The **JVM is the runtime that hosts running programs.**
 - The **JRE is the on-disk part of Java that creates the JVM and loads programs into them.**
 - The **JDK provides the tools necessary to write Java programs that can be executed and run by the JVM and JRE.**
- Developers new to Java often confuse the Java Development Kit and the Java Runtime Environment. The distinction is that **the JDK is a package of tools for developing Java-based software,** whereas the JRE is a package of tools for running Java code.



The figure below shows how the JDK fits into the Java application development lifecycle:



Downloading the JDK

- **Oracle Corporation develops and distributes the Java Development Kit (JDK).**
- **OpenJDK** is the free, open-source implementation of Java and serves as the foundation for many Java distributions.
- **JDK 26** is the latest feature release.
- **JDK 25** is the **latest Long-Term Support (LTS)** release and is the version we will use in this workshop.
- Java is updated regularly. As you continue learning, it's a good idea to check the Oracle website for the latest releases and new LTS versions.
- The JDK is available for:
 - Microsoft Windows
 - macOS
 - Linux



Downloading the JDK (Windows ver.)

- The first step is to download the Oracle Java Development Kit (JDK) from the official Oracle website.
- Visit:
 - <https://www.oracle.com/java/technologies/downloads/>
- Select JDK 25 (Long-Term Support).
- Choose the installer that matches your computer:
 - **Windows x64 Installer (.exe) (most Windows computers)**
 - **Windows ARM64 Installer (.msi) (Windows on ARM devices, such as some Surface Pro models)**
 - Note: Most Windows computers use the x64 installer. If you're unsure, choose x64 or ask for assistance.
- Download the installer and follow the installation wizard.



Downloading the JDK (Windows ver.)

- Now we can check the Java version installed, open Command Prompt and enter the following commands:

```
java -version  
javac -version
```

- You should see output indicating that Java 25 is installed.
- Congratulations! You have installed the JDK!



Downloading the JDK (macOS ver.)

- Installing the JDK on macOS is very similar to Windows.
- Download the installer that matches your Mac:
 - **Apple Silicon (M1/M2/M3/M4) → ARM64 (AArch64)**
 - **Intel Mac → x64**

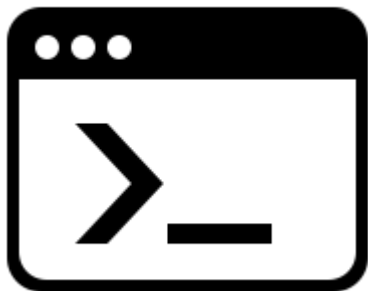


Downloading the JDK (macOS ver.)

- Now we can check the Java version installed, in the terminal, enter the following commands:

```
java -version  
javac -version
```

- You should see output indicating that Java 25 is installed.
- Congratulations! You have installed the JDK!



03

Writing Your First Java Program (Part 1 - Using the CLI)





The Command-line Interface

- When you are starting programming, it may be simpler to use a command-line interface (CLI) for compiling/running.
- You can write Java programs with any text editor (program that lets you edit standard ASCII text files) **saving your file under a .java extension.**
- Unlike C++ programs, but like Scala programs, Java programs don't compile to code that can be executed directly. Instead **they compile to “bytecode” (.class files) meant to be executed by a Java virtual machine (JVM).**

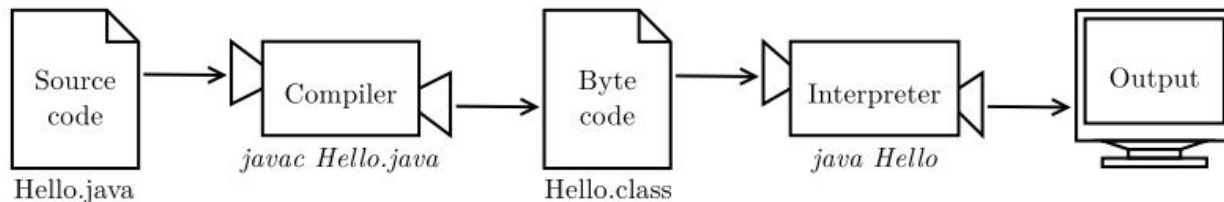


Figure 1.2: The process of compiling and running a Java program.



Writing Our First Program

- Once you have verified that Java is properly installed on your computer, **we can get a simple program running with just a text editor and a command-line interface (CLI) – On a Mac, it's called Terminal, and on Windows, it's Command Prompt**
- We break the process of programming in Java into three steps:
 - a. **Create the program by typing it into a text editor and saving it to a file named, HelloWorld.java.**
 - b. **Compile it by typing "javac HelloWorld.java" in the command-line window.**
 - c. **Execute (or run) it by typing "java HelloWorld" in the command-line window.**
- The first step creates the program; the second translates it into a language more suitable for machine execution (and puts the result in a file named HelloWorld.class); the third actually runs the program.



Writing Our First Program

- **Creating a Java program** – A program is nothing more than a sequence of characters, like a sentence, a paragraph, or a poem.
- To create one, we need only define that sequence characters using a text editor in the same way as we do for email.
- Type the following into your text editor and save it into a file named HelloWorld.java:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        // Prints "Hello, World" in the terminal window.  
        System.out.println("Hello, World");  
    }  
}
```



Writing Our First Program

- **Compiling a Java program** – A compiler is an application that translates programs from the Java language to a language more suitable for executing on the computer.
- It **takes a text file with the .java extension as input (your program) and produces a file with a .class extension (the computer-language version).**
- To compile HelloWorld.java type the following text below into the command-line:

```
javac HelloWorld.java
```

- **If you typed in the program correctly, you should see no error messages.** Otherwise, go back and make sure you typed in the program exactly as it appears above.



Writing Our First Program

- **Executing (or running) a Java program** – Once you compile your program, you can execute it.
- This is the exciting part, where the computer follows your instructions.
- To run the HelloWorld program, type the following in the command-line window:

```
java HelloWorld
```

- If all goes well, you should see the following response:

```
Hello, World
```



How Does the "Hello, World!" Program Work?

- In Java, **every application begins with a class definition.**
- In this particular program, HelloWorld is the name of the class and **the name of the class should match the filename in Java.**
- Next is the main method.
- **Every application in Java must contain the main method.**
- The Java compiler starts executing the code from the main method, and **it's mandatory in each of our executable Java programs.**
- The signature of the main method in Java is:
 - **public static void main(String [] args) { ... }**



How Does the "Hello, World!" Program Work?

- After that, we have a comment.
- In Java, **any line starting with // is a single-line comment.**
- Comments are intended for users reading the code to understand the intent and functionality of the program.
- **It is completely ignored by the Java compiler (an application that translates Java program to Java bytecode that computer can execute).**
- Lastly, we have a **print statement.**
- It **prints the text "Hello, World!" to standard output (your screen).**
- The text inside the quotation marks is called a String literal in Java.
- Notice the print statement is inside the main function, which is inside the class definition.



Creating Your Own Java Program

- For the time being, all of our programs will be just like HelloWorld.java, except with a different sequence of statements in main().
- The easiest way to write such a program is to:
 - **Copy HelloWorld.java into a new file whose name is the program name followed by .java**
 - **Replace HelloWorld with the program name everywhere**
 - **Replace the print statement by a sequence of statements**
- Let's make a new program called **Greeting.java**, that prints two lines:
 - First: **Hello, my name is [YOUR NAME HERE].**
 - Second: **How are you?**



Types of Errors

- There are three main types of errors that can occur in a Java program:
 - **Compile-time (syntax) errors:** detected by the compiler before the program runs and prevent the program from compiling and are usually caused by mistakes in the code's syntax (for example, a missing semicolon or misspelled keyword).
 - **Run-time errors:** occur while the program is running. They happen when the program attempts an **invalid operation** (for example, dividing by zero or accessing an element outside the bounds of an array).
 - **Logical errors:** do not prevent the program from compiling or running, but they **cause it to produce incorrect results**.
Logical errors are often the **most difficult to find and fix**.
- Learning to identify and fix errors is one of the most important programming skills. As you gain experience, you'll also learn how to write code that avoids many common errors.



Types of Errors

- Example: Let's say I typed in the following program:

```
public class Hello {  
    public static void main() {  
        System.out.println("Doesn't execute");  
    }  
}
```

- It compiles fine, but **when I execute it, I get the error `java.lang.NoSuchMethodError: main`**. What am I doing wrong? What type of error is this?



String [] args and User Interactivity

- Typically, we want to provide input to our programs: data that they can process to produce a result.
- The simplest way to provide input data is by using the “String [] args” parameter as illustrated in UseArgument.java below:

```
public class UseArgument {  
  
    public static void main(String[] args) {  
        System.out.print("Hi, ");  
        System.out.print(args[0]);  
        System.out.println(". How are you?");  
    }  
  
}
```



String [] args and User Interactivity

- The **"String[] args"** parameter allows command-line arguments to be passed to the program. (Through the use of an array which you will learn more about later)
- These arguments can be accessed within the main method using the **"args"** parameter.
- **args[0]** is the first (and in our case only) command-line argument that we will pass to the program
- Whenever **this program is executed**, it reads the command-line argument that you type after the program name and prints it back out to the terminal as part of the message.
 - Compilation: **javac UseArgument.java**
 - Execution: **java UseArgument [your name]**



New Java Programming

- For many years, the traditional Java "Hello, World!" program looked like our first example.
- However, in modern Java (starting with Java 21 in preview mode, and finalized in Java 25), **the main method no longer needs to be public static, and the String[] argument is optional**
- This means that we can actually rewrite our HelloWorld.java program to look like this now:

```
void main() {  
    IO.println("Hello, World!");  
}
```

- **You don't even need a class.** Just put `void main() { ... }` in a Java file.



New Java Programming

- To use the command line, **invoking a Java program has become simpler. There is no need to compile the program.** Simply call:

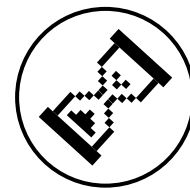
```
java MyProgram.java
```

- The program automatically compiles and runs, just like with Python.
- In this workshop, we'll continue using javac followed by java so you understand the compilation process.



04

Introduction to IDEs (Integrated Development Environments)





What is an IDE?

- Although as we just saw, we can write, compile, and run our programs with a text editor and command-line interface, it may be simpler to use an IDE.
- **A Java IDE (integrated development environment) is a software program that developers use to write and debug code more easily.**
- You are encouraged to develop programs with an IDE (Eclipse, Netbeans, Visual Studio, etc.) because **IDEs have many built-in features that make it easier to work with large programs!**
- Most Java IDEs have a:
 - **code editor,**
 - **a set of tools for automating the building process,**
 - **and a debugger**
- Java IDEs can **increase productivity by combining capabilities such as editing, building and testing within a single application.**



What is an IDE?

- You are free to choose one (or more!) IDE(s) that you like best but as a suggestion some of the most frequently used IDEs include:
 - Eclipse
 - NetBeans
 - IntelliJ IDEA Community Edition
 - DrJava
 - Visual Studio Code (with extensions for Java!)
- Since **NetBeans is slightly easier to use than Eclipse, it's also a good choice for beginner developers.**
- NetBeans has built-in support for Apache Maven projects.



Installing Platform-Independent NetBeans on Windows/macOS

- Since we have already installed the JDK, we can now download Apache NetBeans from:
<https://netbeans.apache.org/front/main/index.html>
- There are several download options available. For this workshop, choose the **Platform Independent ZIP version** (e.g., netbeans-30-bin.zip).
- NetBeans is written in Java, so the same ZIP package works on Windows, macOS, and Linux.
- Extract (unzip) the downloaded archive to a folder of your choice.



05



Writing Your First Java Program (Part 2 - Using the IDE)





Writing Java Programs in NetBeans

- Launch NetBeans from the extracted bin folder:
 - **Windows: Run netbeans64.exe (or netbeans.exe).**
 - **macOS: Run the netbeans executable.**
- For each Java application, you will create a **project** to organize your source files, classes, and other resources:
 - From "**File**" menu ⇒ Choose "**New Project ...**".
 - The "**Choose Project**" dialog pops up ⇒ Under "**Categories**", choose "**Java with Maven**" ⇒ Under "**Projects**", choose "**Java Application**" ⇒ "**Next**".
 - The "**Name and Location**" dialog pops up ⇒ Under "**Project Name**", enter "**FirstJavaProject**" ⇒ In "**Project Location**", select a suitable directory to save your files ⇒ In Group Id: enter "**com.nowhere**" (for example) ⇒ **Finish**.
 - **A starter Java program named FirstJavaProject.java is created automatically.** Right-click on the source file ⇒ **Run File**.



Writing Java Programs in NetBeans

- Now we can try rewriting the programs that we wrote using the text editor and compile and run them through our IDE.
- **Right-click on package "com.nowhere.firstjavaproject" ⇒ New ⇒ Java Class ⇒ In "Class Name", enter "HelloWorld" ⇒ "Finish".**
- The source file "**HelloWorld.java**" appears in the editor panel.
- You can now rewrite the HelloWorld.java file that we previously wrote.
- Once you have done that, there is no need to "compile" the source code in NetBeans explicitly, as NetBeans performs the so-called incremental compilation (i.e., the source statement is compiled as and when it is entered).
- **To run the program, right-click anywhere in the source (or from the "Run" menu) ⇒ Run File.**
- Observe the output on the output console.



Writing Java Programs in NetBeans

- Note: **You should create a NEW Java project for EACH of your Java applications.**
- Nonetheless, **NetBeans allows you to keep more than one programs in a project**, which is handy for writing toy programs (such as your tutorial exercises).
- To run a particular program, open and **right-click on the source file ⇒ Run File.**
- Let's try this by creating a new Java class for **Greeting.java**, rewriting our original Greeting.java code and running it in our IDE.



Writing Java Programs in NetBeans

- Let's create a new Project for UseArgument
- From **"File"** menu $\Rightarrow$ Choose **"New Project ..."**.
- The **"Choose Project"** dialog pops up $\Rightarrow$ Under **"Categories"**, choose **"Java with Maven"** $\Rightarrow$ Under **"Projects"**, choose **"Java Application"** $\Rightarrow$ **"Next"**.
- The **"Name and Location"** dialog pops up $\Rightarrow$ Under **"Project Name"**, enter **"UseArgument"** $\Rightarrow$ In **"Project Location"**, select a suitable directory to save your files $\Rightarrow$ In Group Id: enter **"com.nowhere"** $\Rightarrow$ **Finish**.
- Just like before, a starter Java program named UseArgument.java is created automatically.
- **Delete the code and rewrite our previous UseArgument.java code in its place**



Writing Java Programs in NetBeans

- If you try running the project by pressing the green triangle in the toolbar (which we can since we only have one file in this particular project), you should encounter the following error:

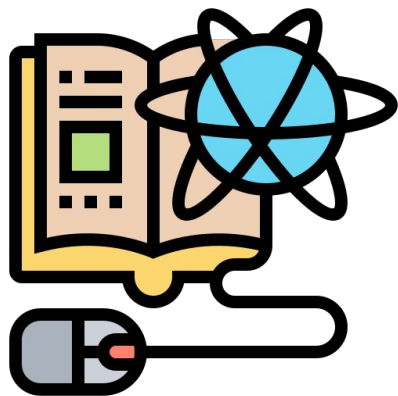
```
Hi, Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0
```

- This is because command-line arguments are normally used in the command-line interface (hence the name)
- However, we can adjust for this in our IDE as well
- **Right-click (on Windows) or Control-Click (on a Mac) the project branch in the NetBeans Projects pane.**
- In the resulting context menu, **select Properties.**



Writing Java Programs in NetBeans

- As a result, the Project Properties dialog box opens.
- On the left side of the Project Properties dialog box, **select Run.**
- In the main body of the Project Properties dialog box, **make sure that the Main Class field contains the**
- **name of the class containing the main method.**
- Then **type the command-line arguments in the Arguments field** (your name).
- **Click OK** to dismiss the Project Properties dialog box.
- **Run the program** in the usual way (by clicking the green arrow Run Project icon in the NetBeans toolbar) and **you should see the same outcome as we did in the CLI.**



06

Overview and Next Steps

NEXT STEPS



Overview

- As of now, you should have/be:
 - Downloaded the Java Development Kit (JDK)
 - Briefly introduced to using the command-line Interface (CLI) to run programs by:
 - Writing simple Java programs in a text editor of your choice
 - Compiling the program using `javac`
 - Executing the program using `java`
 - Downloaded an Integrated Development Environment (IDE) – (we used NetBeans in this workshop but you are free to use whichever you would like)
 - Rewrote and ran our programs using the IDE for a brief introduction to the application



Next Steps

- This all just scratches the surface of all that you will learning in your upcoming CISC 1115 course, but it is definitely a good start!
- You are highly encouraged to read the following text (one of the best intro books for Java AND available as a free PDF online):

Allen Downey and Chris Mayfield, *Think Java: How to Think Like a Computer Scientist*, 2nd Edition, Version 7.1.0, Green Tea Press, 2020, Creative Commons License.



- In addition to whatever resources that will be required from your respective instructors



Next Steps

- I would encourage you to get a feel of **writing programs by hand (your exams and final will all be handwritten)** before running your programs on your own computers
- This, along with **tracing already written programs to determine the output**, will help you better retain information and an understanding of how exactly your programs work
- Furthermore, **tutors are available in the Learning Center – 1300 Boylan Hall** if you ever need any study assistance throughout the course
- Congratulations and best of luck on your CS journey! 🎉

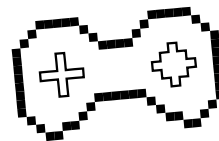


Thank You!

Presented By:

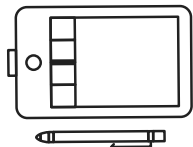
Amara Auguste

**CS Tutor, Graduate Student,
and Adjunct Lecturer**



Do you have any further questions?

amara.auguste@brooklyn.cuny.edu



amaraauguste.github.io